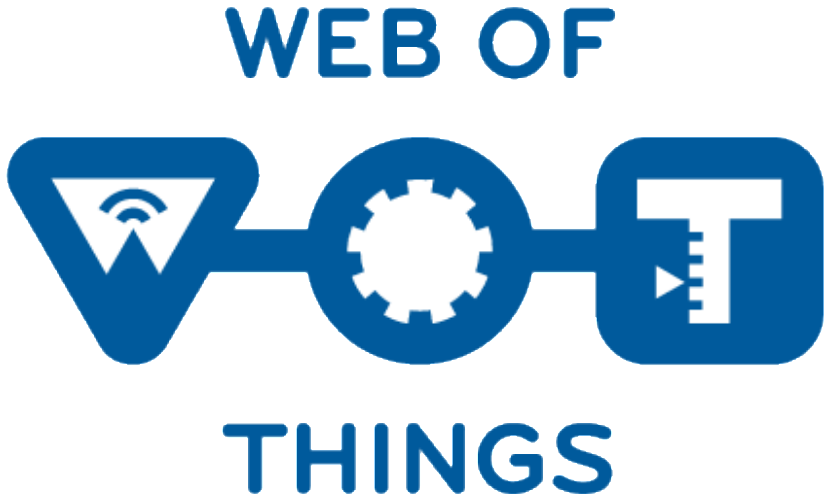


Media Streaming in Thing Description

W3C TPAC2026 Breakout – Media for WoT

Kunihiko Toumura (Hitachi)

2026-10-30

Overview / Plugfest

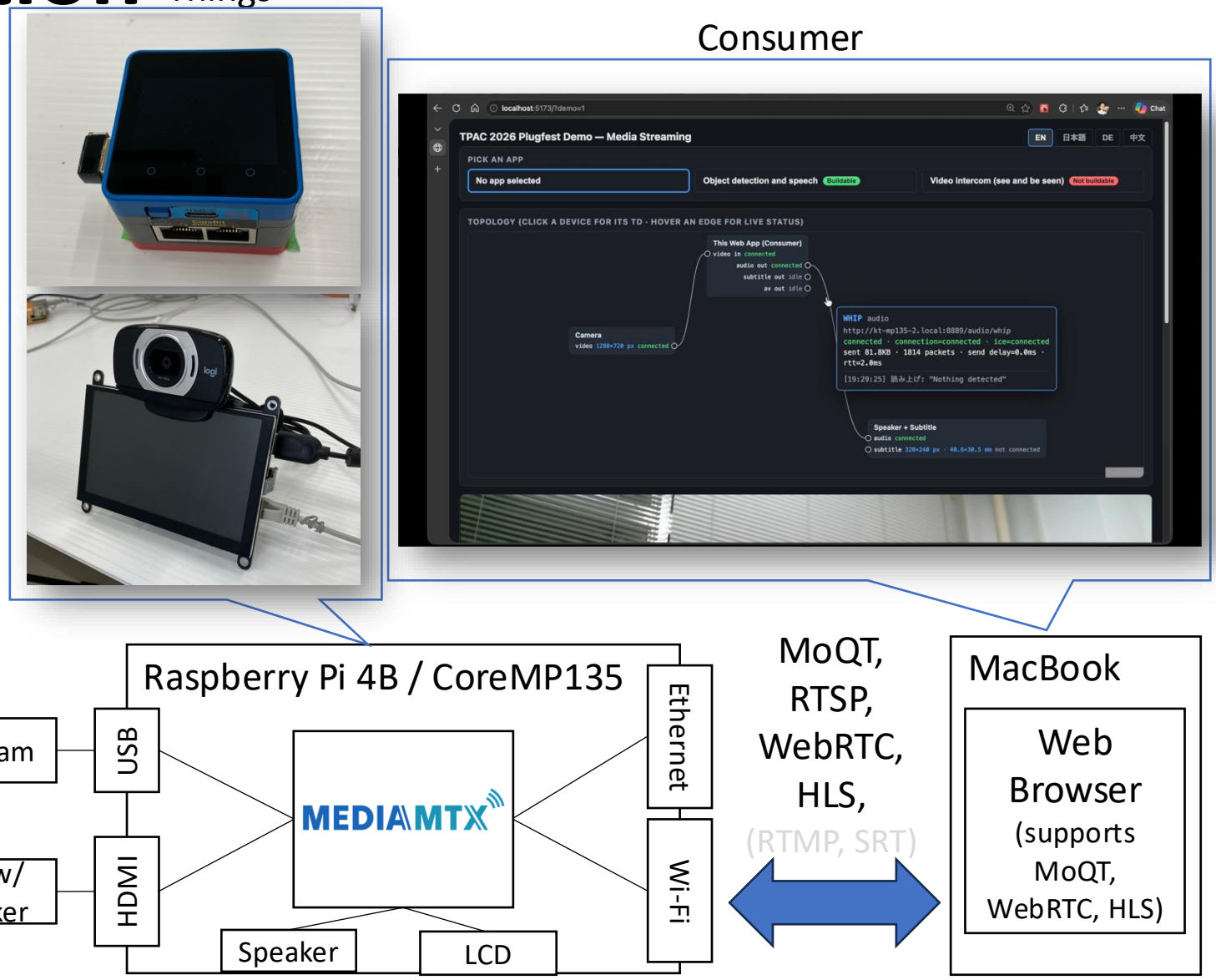
- WoT Thing Description (TD) can describes interactions with devices as "properties", "actions", and "events"
- These assume a simple request-response pattern, or a connection per request with continuous responses
- Media streaming needs different patterns:
  - Data exchanged over a separate connection from the request
  - Multiple tracks (audio, video, data) in a single connection
  - Bidirectional connection
  - Control of an established connection (stop, resume)
- In the Plugfest, we used MediaMTX[1], software that relays various media streams, as a case study to explore how a Thing that handles media streams should be described.

[1] MediaMTX: ready-to-use, open source, live media router, https://mediamtx.org/

```
{
    "@context": [
        "https://www.w3.org/2022/wot/td/v1.1",
        { "saref": "https://w3id.org/saref#" }
    ],
    "id": "urn:uuid:300f4c4b-ca6b-484a-88cf-fd5224a9a61d",
    "title": "MyLampThing",
    "@type": "saref:LightSwitch",
    "securityDefinitions": {
        "basic_sc": {"scheme": "basic", "in": "header"}
    },
    "security": "basic_sc",
    "properties": {                                          Property
        "status": {
            "@type": "saref:OnOffState",
            "type": "string",
            "forms": [{
                "href": "https://mylamp.example.com/status"
            }]
        }
    },
    "actions": {                                             Action
        "toggle": {
            "@type": "saref:ToggleCommand",
            "forms": [{
                "href": "https://mylamp.example.com/toggle"
            }]
        }
    },
    "events": {                                              Event
        "overheating": {
            "data": {"type": "string"},
            "forms": [{
                "href": "https://mylamp.example.com/oh"
            }]
        }
    }
}
```

Plugfest Configuration

- Things: send and receive video and audio using MediaMTX
  - e.g., send camera video, show received video on a display
  - Supported protocols: MoQT, RTSP, RTMP, WebRTC, SRT, HLS
- Consumer: browser application
  - Collects TDs of nearby Things and combines them into apps, such as object detection with audio/video output, or video calling
  - Selects endpoints usable in a browser (MoQT, WebRTC, HLS) and recommends a configuration that meets the app's latency requirements
- Question: To build a consumer like this, what information a Thing Description should contain?



Demo



Media Description Examples in TD (1/3)

Audio/Video Sink (e.g. Digital signage that have LCD display and speaker)

```
{
  "@context": [...], "id": "...", "title": "Camera + Display", "description": "...", ...
  "properties": {
    "avSink": {
      "@type": "wotmedia:Streaming",
      "title": "Audio + video input", "description": "...",
      "writeOnly": true,
      "wotmedia:medium": "wotmedia:AudioVideo",
      "streamcap:accepts": [{
        "streamcap:video": { "streamcap:codec": "avc1.42E01F" },
        "streamcap:audio": { "streamcap:codec": "opus", "streamcap:sampleRate": 48000,... }
      }],
      "forms": [ {
        "op": "writeproperty",
        "href": "http://example.com:8889/display/whip",
        "contentType": "application/sdp",
        "subprotocol": "wotmedia:whip",
        "htv:methodName": "POST",
        "streamcap:receiveToPresentMeanMs": 260,
        "streamcap:receiveToPresentMaxMs": 300,
        "wotmedia:sessionResource": "wotmedia:FromLocationHeader"
      },{
        "op": "writeproperty",
        "href": "moqt://emitter.local:8892/display/",
        "streamcap:receiveToPresentMeanMs": 290,
        "streamcap:receiveToPresentMaxMs": 300,
      }]
    },
```

One property contains multiple tracks (video and audio)

Describes the stream content it accepts, using Media Capabilities / WebCodecs Codec Registry vocabularies

Each forms carry the information a Consumer needs to choose a connection

Media Description Examples in TD (2/3)

Video Source (e.g. Security Camera)

```
"videoStream": {
  "@type": "wotmedia:Streaming",
  "title": "Live video stream",
  "readOnly": true,
  "observable": false,
  "wotmedia:medium": "wotmedia:Video",
  "wotmedia:streamId": "cam",
  "streamcap:codec": "avc1.640028",
  "streamcap:width": 1280,
  "streamcap:height": 720,
  "streamcap:framerate": 30,
  "streamcap:bitrate": 2000000,
  "streamcap:keyFrameIntervalFrames": 30,
  "forms": [ {
    "op": "readproperty",
    "href": "http://example.com:8889/cam/whep",
    "contentType": "application/sdp",
    "subprotocol": "wotmedia:whep",
    "htv:methodName": "POST",
    "streamcap:captureToSendMeanMs": 180,
    "streamcap:captureToSendMaxMs": 210,
    "wotmedia:sessionResource": "wotmedia:FromLocationHeader"
  },
```

Receiving video from a Thing can be described the same way, using a `readOnly` property

```
    {
      "op": "readproperty",
      "href": "http://example.com:8888/cam/index.m3u8",
      "contentType": "application/vnd.apple.mpegurl",
      "htv:methodName": "GET",
      "streamcap:captureToSendMeanMs": 340,
      "streamcap:captureToSendMaxMs": 600
    },{
      "op": "readproperty",
      "href": "moqt://example.com:8892/cam/",
      "streamcap:captureToSendMeanMs": 200,
      "streamcap:captureToSendMaxMs": 240
    },{
      "op": "readproperty",
      "href": "rtsp://emitter.local:8554/cam",
      "contentType": "application/sdp",
      "streamcap:captureToSendMeanMs": 180,
      "streamcap:captureToSendMaxMs": 210
    }]
  }
},
```

Media Description Examples in TD (3/3)

Stop receiving the video stream

```
  "actions": {
    "endStreamSession": {
      "@type": "wotmedia:StreamControl",
      "title": "End an established WHEP stream session",
      "safe": false,
      "idempotent": true,
      "uriVariables": { "sessionId": { "type": "string" }},
      "forms": [{
        "op": "invokeaction",
        "href": "http://example.com:8889/cam/whep/{sessio      ,
        "contentType": "application/json",
        "subprotocol": "wotmedia:whep",
        "htv:methodName": "DELETE",
        "wotmedia:streamOp": "wotmedia:stopStream",
        "wotmedia:appliesToStream": "cam",
        "wotmedia:sessionResource": "wotmedia:FromLocationHeader"
      }]
    }
  }
}
```

Under consideration:
links between affordances, to describe stream control as actions

Lessons Learned from Plugfest

What we found:

- A simple, unidirectional media stream can be described as a property
- A URI alone is not enough to choose the right form (i.e. endpoint, protocol)
  - WHIP, WHEP, and HLS forms all use http(s) URIs
  - Distinguish them by contentType or subprotocol, or by additional vocabulary
- Encoding, resolution, fps, and latency also help select an endpoint
  - External vocabularies could be reused: Media Capabilities, WebCodecs Codec Registry

Open issues:

- Bidirectional streaming
- Data tracks
- Control of streams

Media Vocabulary: three categories

1. **TD core vocabulary**
   - Must be part of the Thing Description standard (defined in TD 2.0)
   - e.g., associating affordances, sharing a connection, extending subprotocol vocabulary
2. **Media-specific vocabulary**
   - Protocol-independent terms for media streaming
   - e.g., codec names, audio/video characteristics (fps, size, bandwidth) that help a Consumer choose a form
   - Reference external vocabularies, or define in a document separate from TD 2.0
3. **Protocol-specific vocabulary**
   - Terms specific to each protocol
   - Defined in documents registered in the WoT Binding Registry

We are not yet defining the details.
So we want to discuss the direction, based on your use cases.

Summary and discussion

- Web of Things (WoT) Thing Description has limited expressiveness for media streaming, so we need to extend it

- Three categories of vocabulary:
  - core, media-specific, protocol-specific

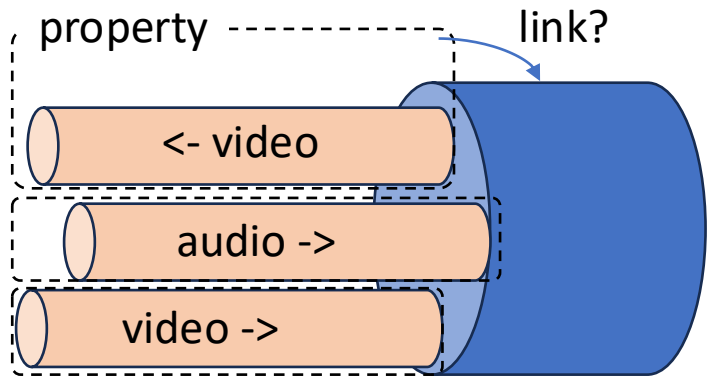
- Discussion: we would like to your concrete use cases and the what vocabulary/ontology you would require

Appendix

TD core vocabulary

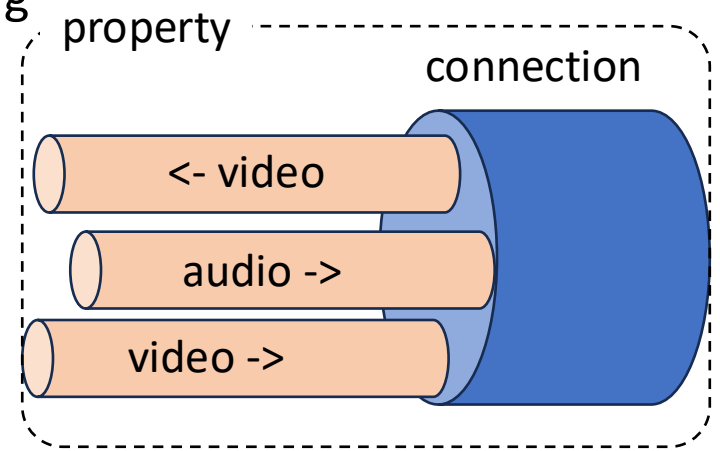
How do we map media onto property, action, and event?

- Bidirectional
- Multiple tracks in a single connection
- Beyond audio and video: time-synchronized data (location, subtitles, image recognition results, etc.)



Directions:

- Option 1: a property = a track
  - One property per track, plus a manageable affordance for creating and deleting the connection, linked in the TD
- Option 2: a property = a connection
  - One property with connection,
    and create/delete described in its forms
- Option 3: Other?



Common Media vocabulary

Many media vocabularies already exist.
Are they sufficient? Some need conversion to OWL?

Vocabulary	Form	Notes
Media Capabilities, Media Capture and Streams, WebCodecs	WebIDL	Not RDF terms https://www.w3.org/TR/media-capabilities/ https://www.w3.org/TR/mediacapture-streams/ https://www.w3.org/TR/webcodecs/
MPEG-DASH MPD, MSF catalog	XML schema / JSON	Not RDF terms ISO/IEC 23009-1:2022, https://datatracker.ietf.org/doc/draft-ietf-moq-msf/01/
EBUCorePlus (ec:)	OWL	Maintained. Successor of EBUCore https://github.com/ebu/ebucoreplus
Ontology for Media Resources 1.0 (ma:)	OWL	W3C Recommendation (2012) https://www.w3.org/TR/mediaont-10/
schema.org (schema:)	OWL	Describes works and files (VideoObject is “a video file”) https://schema.org/

Protocol-specific vocabulary

- Not needed if a Consumer can understand how to access the Thing just from the URI defined by the standard
- If the URI cannot express it, write a WoT Binding document for that protocol and register it in the registry